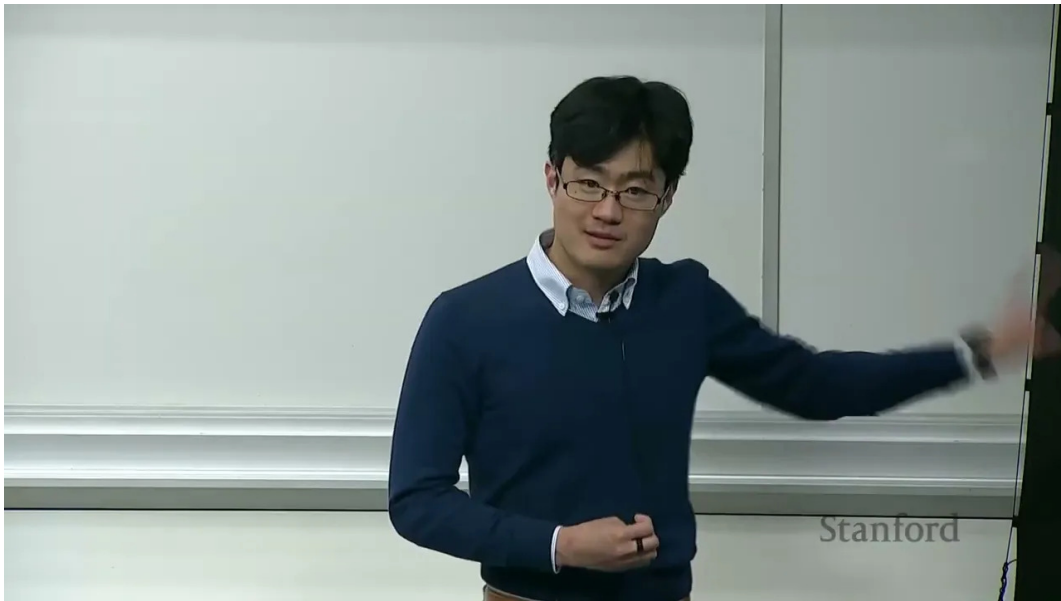


课程讲义

从稠密网络到稀疏专家系统

Stanford CS336 Spring 2025 · Lecture 4 · Mixture of Experts



频道：Stanford Online

发布日期：2025 年 4 月 24 日

视频时长：1 小时 22 分 04 秒

视频链接：<https://www.youtube.com/watch?v=LPv1KfUXLCo>

讲义语言：中文；公式、代码与术语保留英文符号以便对照实现。

整理说明：依据完整人工英文字幕、原始视频课件页和逐图时间窗校验重构；不是字幕逐句翻译。

整理日期：2026 年 7 月 29 日

目录

1	为什么需要混合专家模型	3
1.1	MoE 的第一性原理	3
1.2	参数、计算与记忆的解耦	4
1.3	本章小结	6
2	架构位置、并行机会与真实代价	6
2.1	专家并行为什么自然	6
2.2	为什么 MoE 没有完全取代稠密模型	7
2.3	MoE 通常替换 FFN，而非注意力	8
2.4	本章小结	10
3	Top-k 路由：主流机制与设计变体	10
3.1	为什么主流收敛到 token-choice top- k	10
3.2	从路由 logits 到专家输出	11
3.3	本章小结	13
4	专家粒度与共享专家	13
4.1	把大专家切成细粒度专家	13
4.2	共享专家隔离公共知识	13
4.3	本章小结	15
5	如何训练不可微的路由器	15
5.1	三条路线：RL、随机近似、启发式	15
5.2	随机扰动与 router jitter	17
5.3	本章小结	19
6	负载均衡：从辅助损失到在线偏置	19
6.1	Switch 风格的专家平衡	19
6.2	从专家平衡扩展到设备平衡	21
6.3	DeepSeek-V3 的辅助损失自由偏置	21
6.4	本章小结	22
7	系统实现、容量与数值稳定性	22
7.1	Expert parallelism 的数据流	22
7.2	容量因子与 token dropping	23

7.3	路由器为什么常用 float32	24
7.4	本章小结	26
8	训练策略与 DeepSeek 系列演进	26
8.1	从稠密模型 upcycle 成 MoE	26
8.2	DeepSeekMoE V1: 细粒度与共享专家	27
8.3	DeepSeek-V2: Top-M 设备约束通信	28
8.4	DeepSeek-V3: 平衡控制与推理鲁棒性	29
8.5	本章小结	30
9	MLA、MTP 与全课总结	30
9.1	MLA: 把 KV cache 压缩到低维潜变量	30
9.2	MTP: 同时预测多个未来 token	31
9.3	把全讲串成一条工程链	32
9.4	复习检查表	33
9.5	本章小结	34

1 为什么需要混合专家模型

1.1 MoE 的第一性原理

标准 Transformer 的每个 token 都会经过同一组注意力层和前馈网络 (FFN)。混合专家模型 (Mixture of Experts, MoE) 保留注意力路径, 却把一个稠密 FFN 替换为多个候选 FFN; 路由器只让每个 token 激活其中少数几个。于是, 模型可以拥有很大的总参数容量, 但单个 token 的激活计算量仍接近一个较小的稠密模型。

若隐藏维度为 d 、FFN 中间维度为 d_{ff} , 忽略偏置和激活函数, 单个稠密 FFN 的矩阵乘法量可近似写成

$$C_{\text{dense}} \approx 2d d_{\text{ff}}.$$

- d : Transformer 残差流的隐藏维度;
- d_{ff} : FFN 扩张后的中间维度;
- 系数 2: 对应上投影和下投影两次矩阵乘法。

MoE 层有 N 个专家, 路由器对第 t 个 token 只选择集合 S_t 中的 k 个专家:

$$y_t = \sum_{i \in S_t} g_{t,i} E_i(x_t), \quad |S_t| = k \ll N.$$

- x_t, y_t : 第 t 个 token 进入与离开 MoE 层的表示;
- E_i : 第 i 个专家, 本质上通常是一个独立 FFN;
- $g_{t,i}$: 路由器给该 token-专家对分配的权重;
- S_t : 当前 token 实际激活的专家集合。

一句话抓住 MoE

MoE 不是“每个领域配一个人工命名专家”, 而是“让参数容量稠密增长、让每个 token 的计算保持稀疏”。专家是否形成语义分工是训练结果, 不是架构先验。

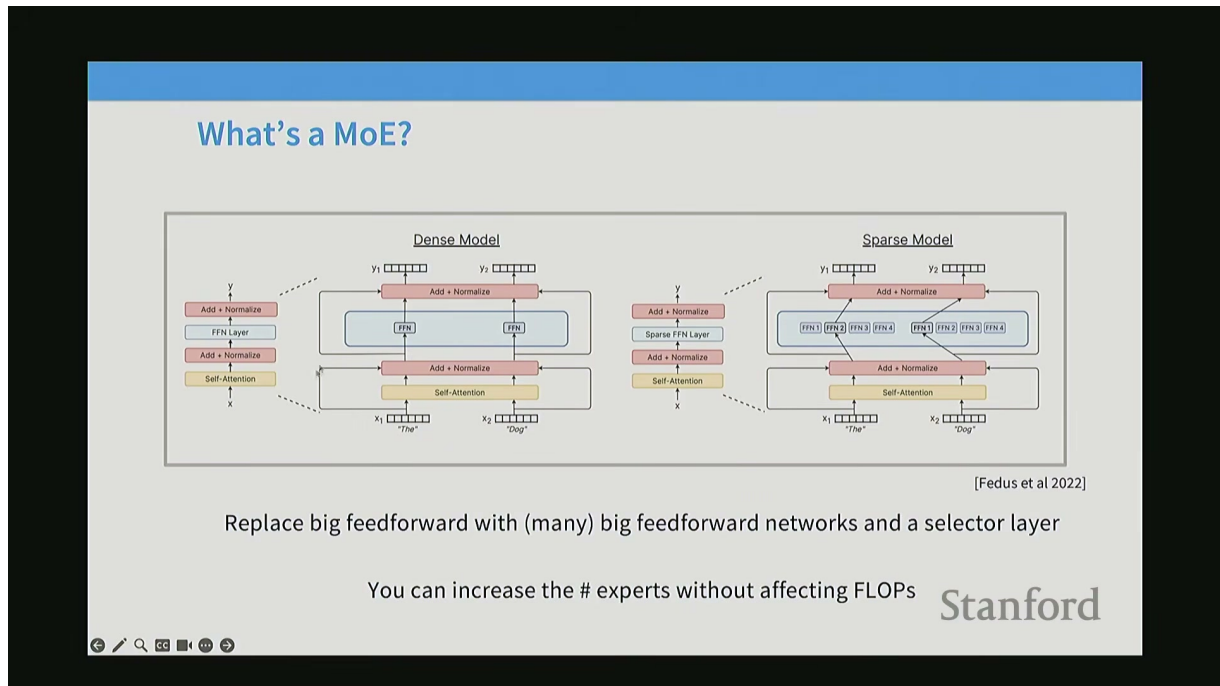


图 1: 稠密 FFN 与稀疏 MoE: 注意力路径基本不变, 主要替换 FFN。
视频画面时间区间: 00:01:55-00:02:35。

1.2 参数、计算与记忆的解耦

假设每个专家与原 FFN 同尺寸, 则专家参数预算约为

$$P_{\text{experts}} \approx N \cdot 2d d_{\text{ff}}.$$

- P_{experts} : 全部专家的参数总量;
- N : 专家总数;
- 该式描述存储容量, 不代表每个 token 都会使用全部参数。

单个 token 的专家计算却只与激活数 k 近似成正比:

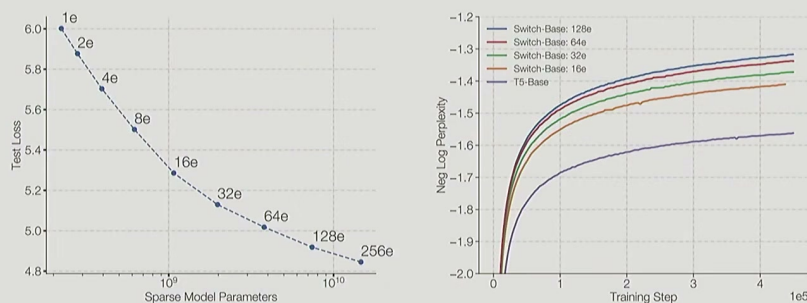
$$C_{\text{MoE}} \approx k \cdot 2d d_{\text{ff}} + C_{\text{router}}.$$

- k : 每个 token 激活的专家数;
- C_{router} : 计算路由得分、选取 top- k 与重排 token 的成本;
- 当 $k \ll N$ 时, 总参数量和激活计算量被显著解耦。

这种解耦解释了固定 FLOPs 下“专家越多、损失越低”的经验曲线: 更多参数可以记忆更多模式, 而每次前向仍只访问少量专家。但曲线是实验规律, 不是对任意规模、任意数据都成立的数学定理; 显存、通信和优化困难可能提前成为瓶颈。

Why are MoEs getting popular?

Same FLOP, more param does better



(Fedus et al 2022)

图 2: 固定计算预算下增加专家数量, 可获得更低测试损失的经验趋势。

视频画面时间区间: 00:03:10-00:03:50。

Why are MoEs getting popular?

Faster to train MoEs

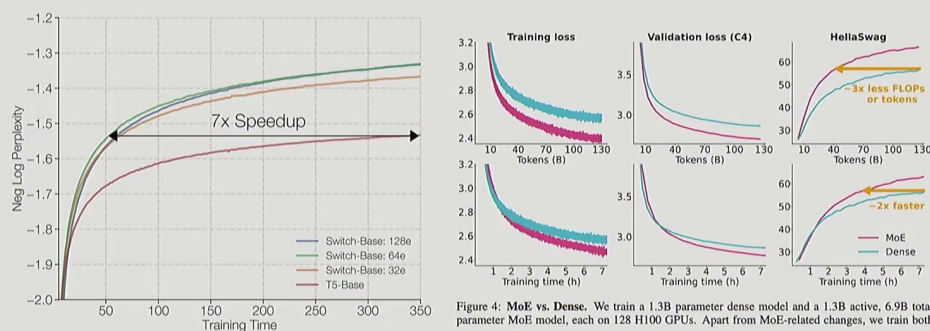


Figure 4: MoE vs. Dense. We train a 1.3B parameter dense model and a 1.3B active, 6.9B total parameter MoE model, each on 128 H100 GPUs. Apart from MoE-related changes, we train both

(Fedus et al 2022)

图 3: MoE 的计算效率案例: 课件同时展示约 7 倍加速与 OLMoE 对比。

视频画面时间区间: 00:05:40-00:06:20。

不要混淆三种“模型大小”

总参数量决定存储和加载成本; 激活参数量描述一次 token 前向使用的参数; 实际 **FLOPs**/吞吐还受路由、通信、kernel 利用率和批次形状影响。用 activated parameters 作横轴时, MoE 往往看起来格外占优, 因此必须同时报告总参数与系统测量。

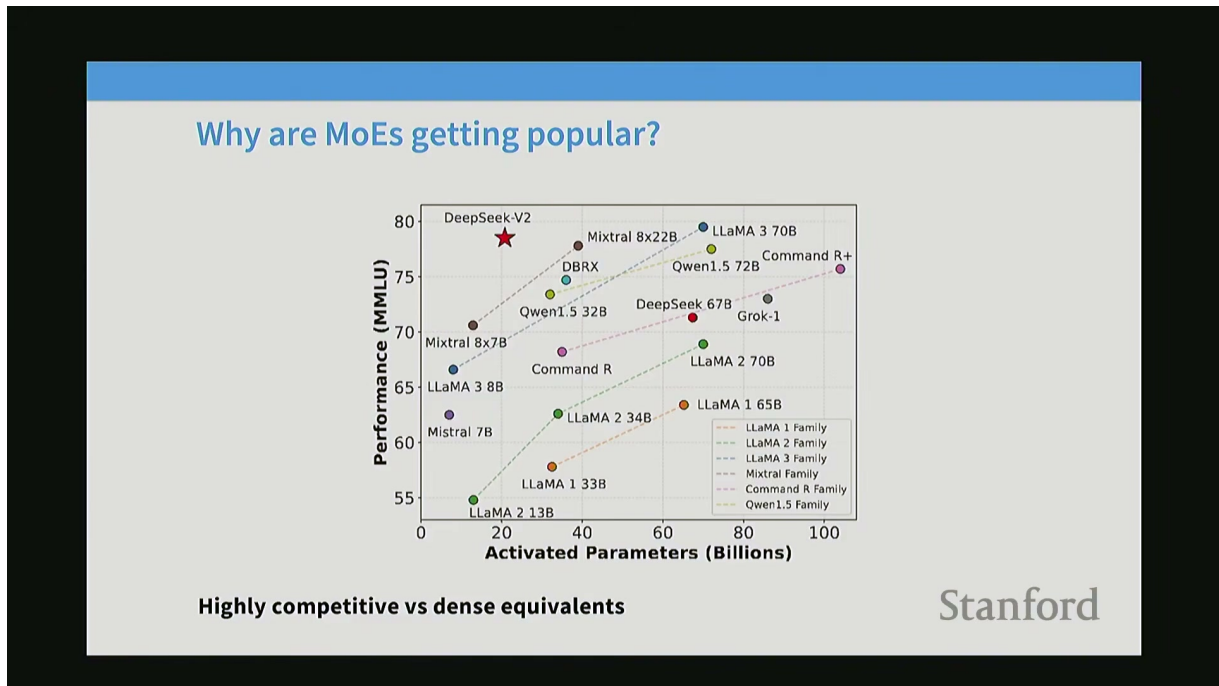


图 4: 以激活参数作横轴的性能图很有吸引力, 但隐藏了总参数和系统成本。

视频画面时间区间: 00:06:55–00:07:35。

1.3 本章小结

MoE 的核心收益来自“容量大、激活稀疏”。它能在近似固定的 token 计算预算下增加参数量, 并改善语言建模损失; 与此同时, 总参数存储、跨设备通信和离散路由训练并不会随 FLOPs 口径一起消失。后续所有设计都围绕一个问题展开: 怎样把这种理论上的稀疏优势转化为可训练、可扩展、可推理的系统优势。

2 架构位置、并行机会与真实代价

2.1 专家并行为什么自然

不同专家拥有彼此独立的 FFN 权重, 因此可以把专家切分到不同设备。路由完成后, token 按专家重新排列并发送到持有该专家的设备; 专家计算结束后, 再把结果发回原 token 顺序。这形成了 expert parallelism (EP), 并能与数据并行、张量并行或流水线并行组合。

若设备 q 持有专家集合 $\mathcal{E}_q$, 其理想负载可写为

$$L_q = \sum_t \sum_{i \in \mathcal{E}_q} \mathbf{1}[i \in S_t].$$

- L_q : 设备 q 在当前批次接收的 token-expert 对数量;
- $\mathcal{E}_q$: 映射到设备 q 的专家集合;
- 指示函数 $\mathbf{1}[\cdot]$: 某专家被该 token 激活时取 1。

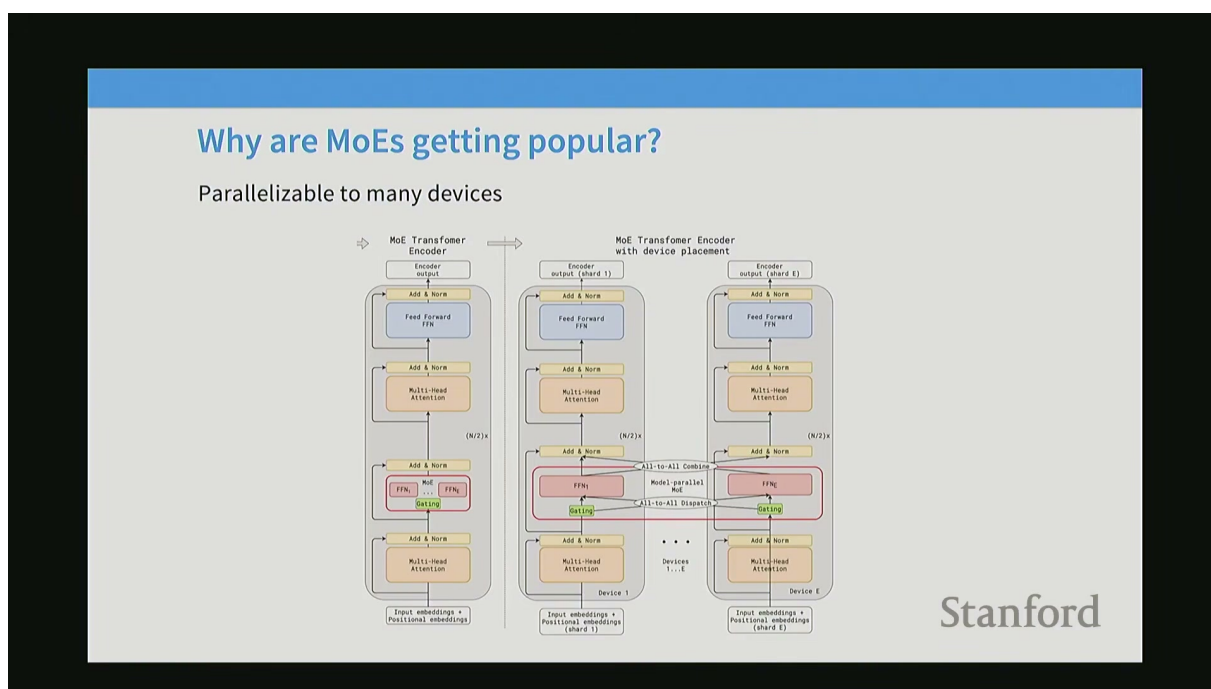


图 5: 专家可分布到不同设备, 但 token 必须经历跨设备 all-to-all。

视频画面时间区间: 00:08:10-00:08:50。

并行优势不是免费午餐

专家权重天然可分片, 适合扩展总参数; 但路由结果依赖输入, 通信模式也是动态的。只要某些专家过热, 最慢设备就会拖住全局同步。MoE 因而把一部分“矩阵乘法难题”转化成了“动态通信与负载控制难题”。

2.2 为什么 MoE 没有完全取代稠密模型

理论 FLOPs 的吸引力非常强, 但实际训练要面对四类额外成本:

- 基础设施复杂度: all-to-all、token permutation、分布式 checkpoint 和容错都更复杂;
- 低算术强度: 细碎专家和不规则 token 数会降低 GPU kernel 利用率;
- 优化不稳定: top- k 是离散选择, 负载平衡主要依赖启发式目标;
- 部署代价: 即使每个 token 只激活少量参数, 服务节点仍可能需要持有或调度全部专家权重。

Why haven't MoEs been more popular?

Infrastructure is complex / advantages on multi node

At a high level, sparsity is good when you have many accelerators (e.g. GPU/TPU) to host all the additional parameters that comes when using sparsity. Typically models are trained using data-parallelism where different machines will get different slices of the training/inference data. The machines used for operating on the different slices of data can now be used to host many more model parameters. Therefore, sparse models are good when training with data parallelism and/or have high throughput while serving: training/serving on many machines which can host all of the parameters.

[Fedus et al 2022]

Training objectives are somewhat heuristic (and sometimes unstable)

Sparse models often suffer from training instabilities (Figure 1) worse than those observed in standard densely-activated Transformers.

Stanford
[Zoph et al 2022]

图 6: MoE 的真实门槛：多节点基础设施和依赖启发式的训练目标。

视频画面时间区间：00:11:55–00:12:35。

吞吐必须在目标硬件上测

“同 FLOPs” 不等于 “同 wall-clock time”。若专家太小、批量太少或网络互连太慢，稀疏计算节省可能被通信和调度开销抵消。模型论文中的理论计算量不能直接替代部署环境的端到端吞吐与尾延迟。

2.3 MoE 通常替换 FFN，而非注意力

经典做法是把 Transformer block 中参数占比较大的 FFN 替换为专家集合。注意力头也可以做稀疏专家化，但它涉及序列内 token 交互和 KV cache，路由与实现更棘手，因此主流大模型通常采用 MLP-MoE。这个边界很重要：本讲后续出现的“专家”若无特别说明，均指 FFN 专家。

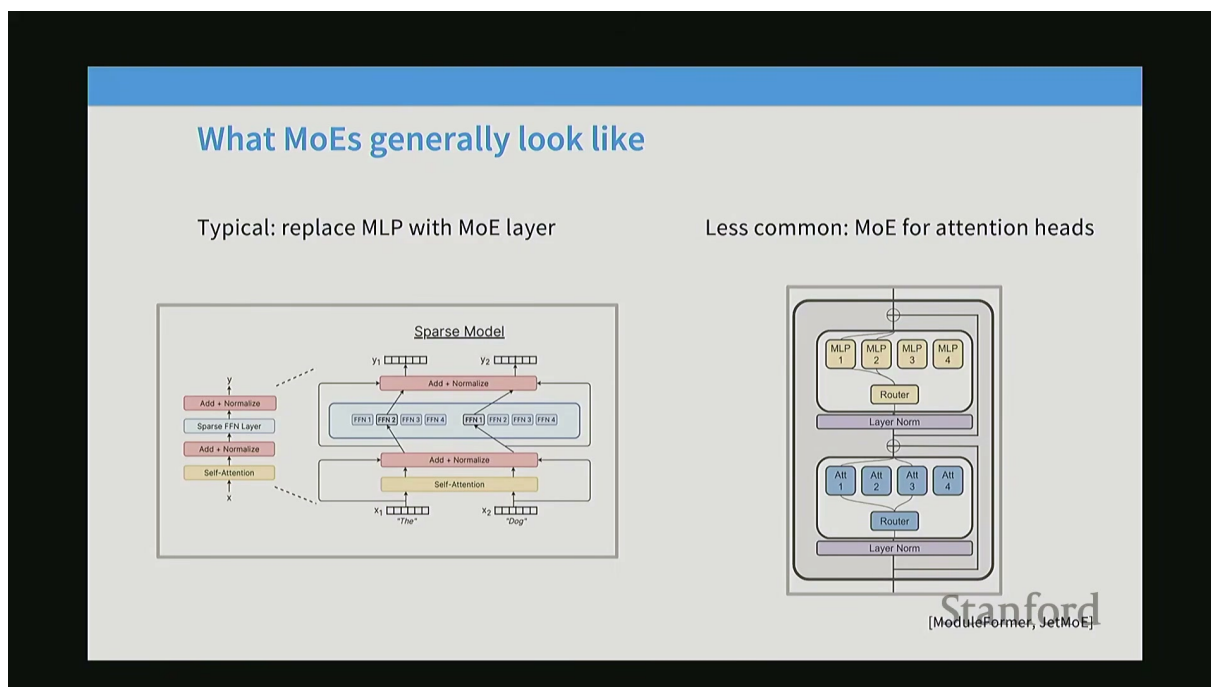


图 7: 主流是 MLP-MoE; 把注意力头做成专家仍属少见设计。

视频画面时间区间: 00:13:40-00:14:20。

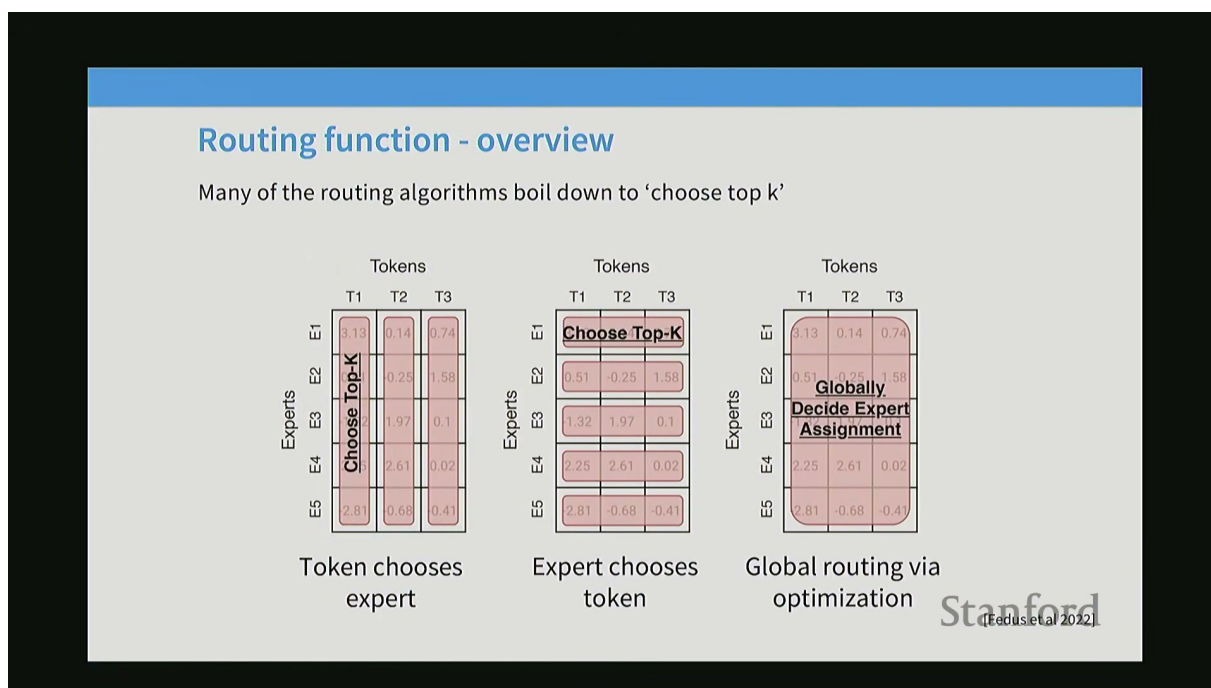


图 8: 三种分配视角: token 选专家、专家选 token、全局联合优化。

视频画面时间区间: 00:15:10-00:15:50。

路由问题的本质

路由不是给专家贴标签, 而是在容量、质量、通信和可微性之间做受约束分配。token-choice 简单且适合在线推理; expert-choice 容易控制每个专家的负载; 全局优化能表达更多约束, 却通常不适合高吞吐自回归生成。

2.4 本章小结

MoE 最自然地落在 FFN, 并通过 expert parallelism 扩展参数。但动态 token 分配引入 all-to-all、负载长尾和不规则 kernel。架构是否成功, 不能只看损失或理论 FLOPs, 还要同时看通信、显存、吞吐、稳定性和服务形态。

3 Top- k 路由: 主流机制与设计变体

3.1 为什么主流收敛到 token-choice top- k

历史上存在 token-choice、expert-choice、哈希路由与全局匹配等路线。主流开放模型最终多采用 token-choice top- k : 每个 token 独立计算对所有专家的 affinity, 再选得分最高的 k 个。这种方式实现直接、可批处理, 也与自回归推理天然兼容。expert-choice 可以精确控制专家容量, 却让一个 token 被选中的次数不固定, 并使在线推理行为更复杂。

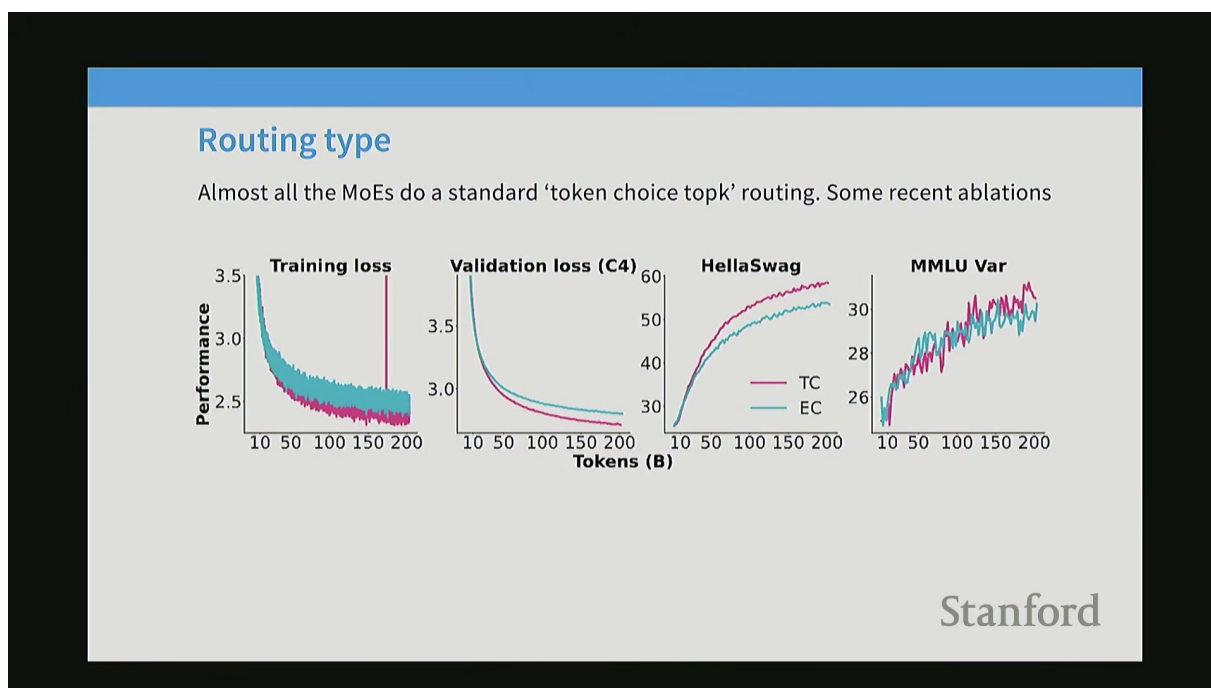
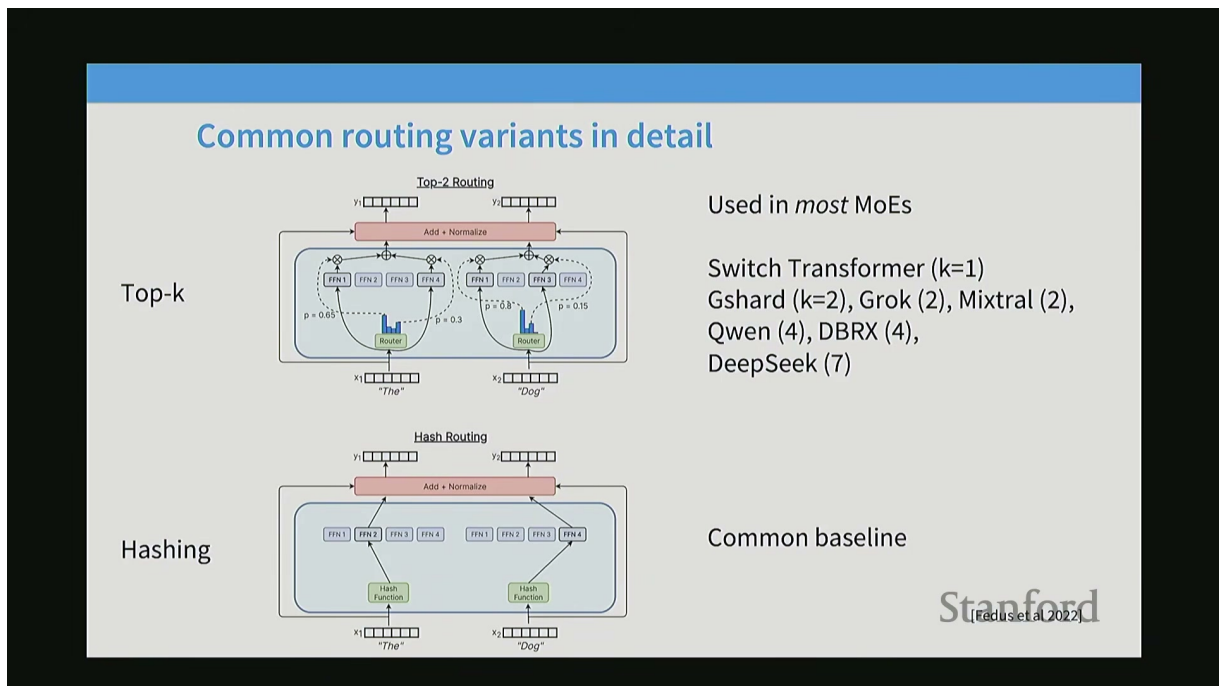


图 9: OLMoE 消融中, token-choice top- k 成为更可靠的主流选择。

视频画面时间区间: 00:16:55-00:17:35。

图 10: Top- k 与哈希路由的设计谱系，以及不同模型采用的 k 。

视频画面时间区间: 00:21:10–00:21:50。

Top-1 与 Top-2 的基本权衡

Top-1 计算最省、通信最少，但单个错误路由无法被第二专家修正；Top-2 或更大 k 往往更稳健，却增加专家计算和 token 复制通信。最优 k 依赖专家粒度、模型规模、互连和质量目标，而不是一个跨模型恒定的数字。

3.2 从路由 logits 到专家输出

最简单的路由器是一层线性映射。对 token 表示 x_t ，第 i 个专家的未归一化得分为

$$s_{t,i} = w_i^\top x_t.$$

- $s_{t,i}$: token t 对专家 i 的 router logit;
- w_i : 专家 i 的路由向量;
- x_t : 当前 token 的隐藏表示。

若先对全部专家做 softmax，则

$$p_{t,i} = \frac{\exp(s_{t,i})}{\sum_{j=1}^N \exp(s_{t,j})}.$$

- $p_{t,i}$: 路由器对专家 i 的归一化偏好;
- N : 路由专家总数;
- 分母包含全部专家，因此未激活专家也参与归一化。

选择集合定义为

$$S_t = \text{TopK}(\{p_{t,i}\}_{i=1}^N, k).$$

- TopK: 返回数值最大的 k 个索引;
- S_t : 真正执行前向的专家索引集合;
- 该集合随 token 而变, 导致稀疏、输入依赖的计算图。

MoE 输出于是为

$$y_t = \sum_{i \in S_t} p_{t,i} E_i(x_t).$$

- $E_i(x_t)$: 专家 i 对 token 表示的变换;
- $p_{t,i}$: 同时承担选择依据和混合权重;
- 未进入 S_t 的专家不执行 FFN 计算。

另一种实现先选 top- k logits, 再只在选中集合内归一化:

$$\tilde{p}_{t,i} = \frac{\exp(s_{t,i})}{\sum_{j \in S_t} \exp(s_{t,j})}, \quad i \in S_t.$$

- $\tilde{p}_{t,i}$: 选中专家内部重新归一化后的权重;
- 该形式使选中权重之和严格为 1;
- softmax-before-top- k 与 softmax-after-top- k 的梯度和尺度并不完全相同。

Top-K routing in detail.

Most papers do the old and classic top-k routing. How does this work?

Gating

$$\mathbf{h}_t^l = \sum_{i=1}^N \left(g_{i,t} \text{FFN}_i(\mathbf{u}_t^l) \right) + \mathbf{u}_t^l,$$

$$g_{i,t} = \begin{cases} s_{i,t}, & s_{i,t} \in \text{Topk}(\{s_{j,t} | 1 \leq j \leq N\}, K), \\ 0, & \text{otherwise,} \end{cases}$$

$$s_{i,t} = \text{Softmax}_i(\mathbf{u}_t^{lT} \mathbf{e}_i^l),$$

↑
Gates selected by a logistic regressor

This is the
DeepSeek (V1-2) router
(Grok, Qwen do this too)

Mixtral, DBRX,
DeepSeek v3
softmaxes after the
TopK



[Dai et al 2024]

图 11: 经典 top- k 路由公式, 以及 softmax 与 TopK 的顺序变体。

视频画面时间区间: 00:24:25-00:25:05。

离散选择的梯度断点

TopK 的索引对微小 logit 变化呈分段常数: 当前被选专家可从混合权重获得梯度, 但“差一点被选中”的专家不会通过离散索引获得普通反向传播信号。这正是后续 RL、随机扰动与负载均衡启发式出现的原因。

3.3 本章小结

token-choice top- k 以简单、可扩展和适合在线推理胜出。标准流程是线性打分、选择少数专家、对其输出加权。softmax 放在 TopK 前还是后、 k 取多大、是否加入共享专家，都会改变梯度、通信与鲁棒性，但都没有消除离散选择本身。

4 专家粒度与共享专家

4.1 把大专家切成细粒度专家

假设原 FFN 的中间宽度为 d_{ff} ，把它切成 m 个较小专家，每个专家宽度约为

$$d_{\text{expert}} = \frac{d_{\text{ff}}}{m}.$$

- d_{expert} : 单个细粒度专家的中间维度;
- m : 等宽切分数;
- 若同时激活多个细粒度专家，可在近似计算量下获得更丰富的组合。

细粒度带来组合选择：不再是“从少数大 FFN 中选一个”，而是“从许多小子网络中选若干个”。同样的激活宽度下，可选组合数增加，路由器有更精细的表达空间。代价是每个专家接收的 token 更少，kernel 更碎，通信元数据也更多。

4.2 共享专家隔离公共知识

共享专家对所有 token 始终激活，路由专家只承担输入相关的增量变换：

$$y_t = E_{\text{shared}}(x_t) + \sum_{i \in S_t} g_{t,i} E_i(x_t).$$

- E_{shared} : 始终执行、承载通用能力的共享专家;
- E_i : 按 token 选择的路由专家;
- 共享路径降低所有路由专家重复学习公共模式的压力。

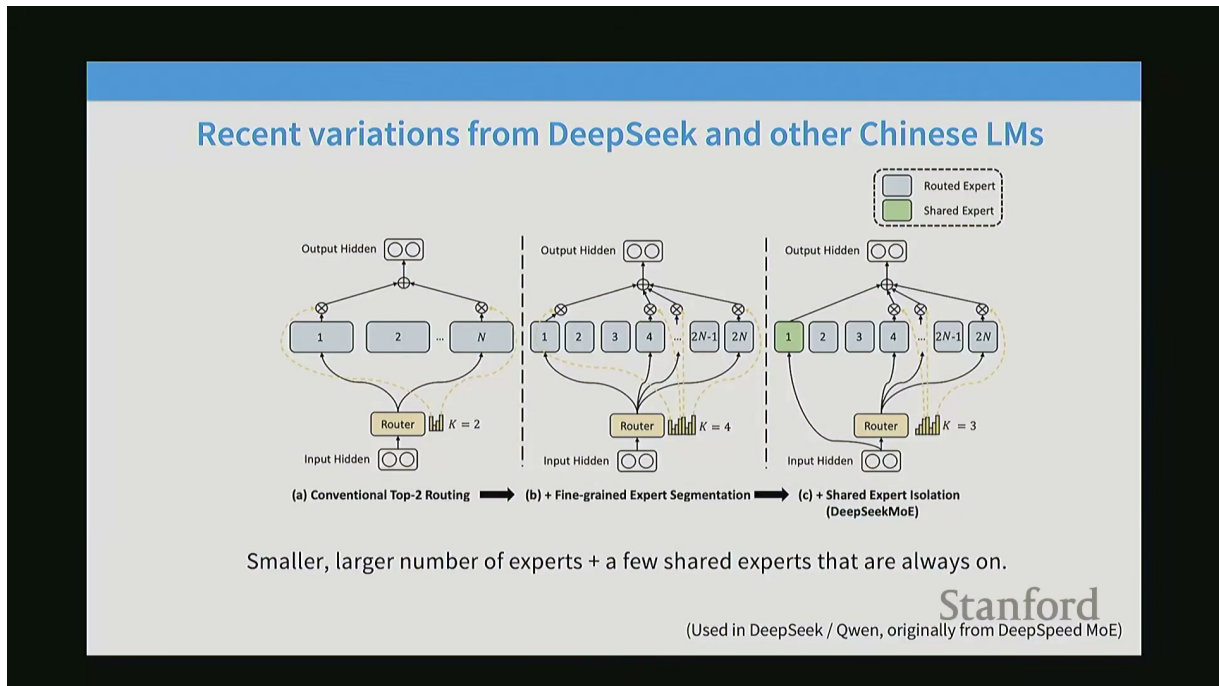


图 12: 细粒度专家分割与共享专家隔离: DeepSeek 系列的重要架构选择。

视频画面时间区间: 00:31:55–00:32:35。

细粒度的收益更稳定, 共享专家仍需消融

课件强调, 开放 MoE 发布普遍采用细粒度专家, 相关消融收益较清晰; 共享专家的第三方复现结果更混合。工程上应把二者视为独立旋钮, 分别控制总专家数、激活宽度和共享路径, 而不是绑定成不可拆分的“DeepSeek 配方”。

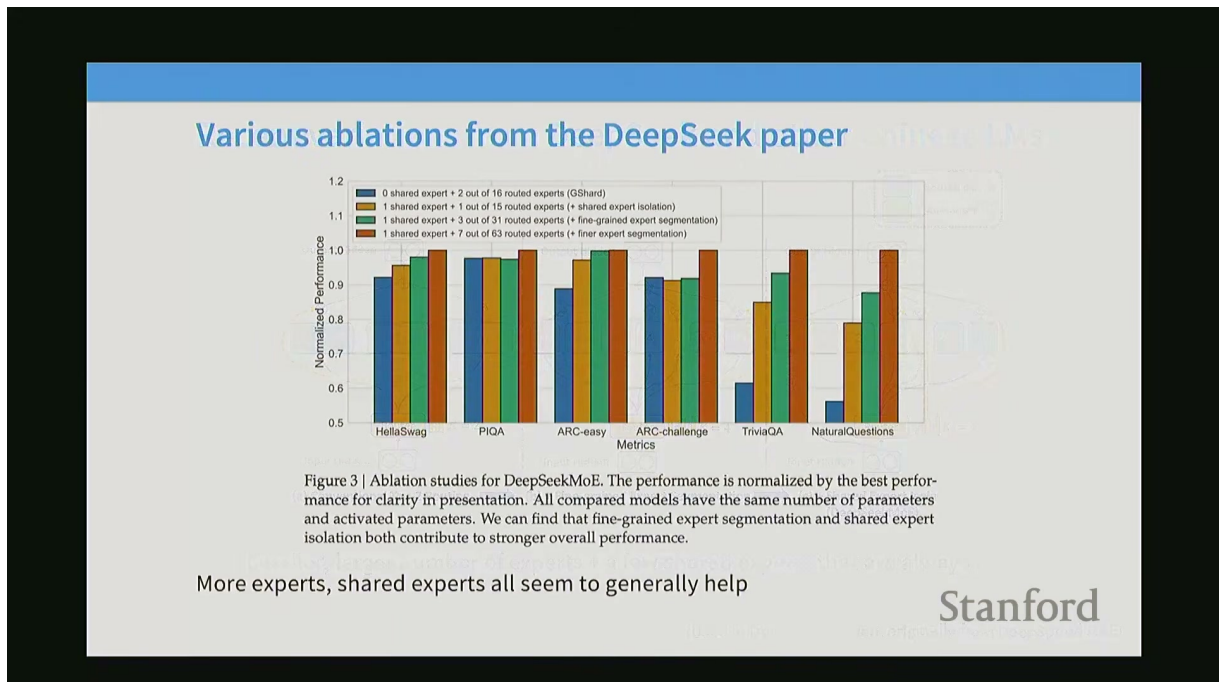


图 13: DeepSeek 消融: 更多、更细的专家通常有益, 共享专家收益需结合复现判断。

视频画面时间区间: 00:34:25–00:35:05。

Expert routing setups for recent MoEs				
Model	Routed	Active	Shared	Fine-grained ratio
GShard	2048	2	0	
Switch Transformer	64	1	0	
ST-MOE	64	2	0	
Mixtral	8	2	0	
DBRX	16	4	0	
Grok	8	2	0	
DeepSeek v1	64	6	2	1/4
Qwen 1.5	60	4	4	1/8
DeepSeek v3	256	8	1	1/14
OLMoE	64	8	0	1/8
MiniMax	32	2	0	~1/4
Llama 4 (maverick)	128	1	1	1/2

图 14: 近期 MoE 配置跨越不同总专家数、激活数、共享数与细粒度。

视频画面时间区间: 00:36:40–00:37:20。

设计时优先固定“激活计算”

比较专家粒度时，应先固定每个 token 的总激活 FFN 宽度，再改变专家数量与单专家宽度。否则质量变化可能只是来自计算量变化。随后再用系统测试确认小专家是否仍能形成足够大的 GEMM，避免理论等算力、实际低利用率。

4.3 本章小结

细粒度专家通过更多组合提升路由表达力，共享专家试图隔离普通知识。前者的实证支持更一致，后者应按数据和模型逐项消融。设计比较必须同时固定激活计算、报告总参数，并测量专家 kernel 与通信效率。

5 如何训练不可微的路由器

5.1 三条路线：RL、随机近似、启发式

路由器面临两种不同的学习信号：一是“把 token 送给哪个专家能降低语言模型损失”，二是“不要把所有 token 都送给同一个专家”。由于 TopK 不可微，研究者尝试过：

1. 用 REINFORCE 等策略梯度直接优化离散动作；
2. 给 logits 注入可微随机扰动，以平滑或探索路由边界；
3. 保留普通反向传播，再加入负载平衡辅助损失。

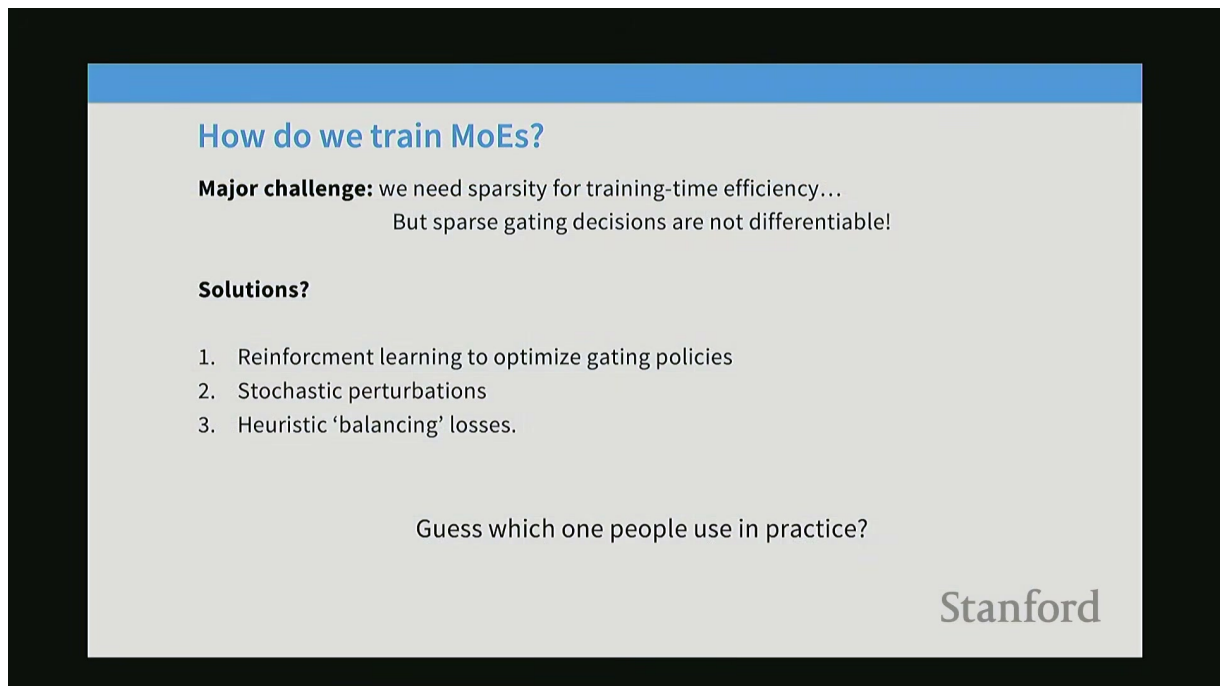


图 15: 稀疏离散路由的三类训练路线。

视频画面时间区间: 00:41:25–00:42:05。

策略梯度估计可以写为

$$\nabla_{\theta} J(\theta) = \mathbb{E}_{a \sim \pi_{\theta}(\cdot|x)} [(R - b) \nabla_{\theta} \log \pi_{\theta}(a|x)] .$$

- θ : 路由策略参数;
- a : 离散专家选择动作;
- R : 由语言模型损失构造的回报;
- b : 降低方差的 baseline。

它在理论上直接对应离散决策, 却常因方差大、实现复杂、收益不稳定而不如启发式实用。

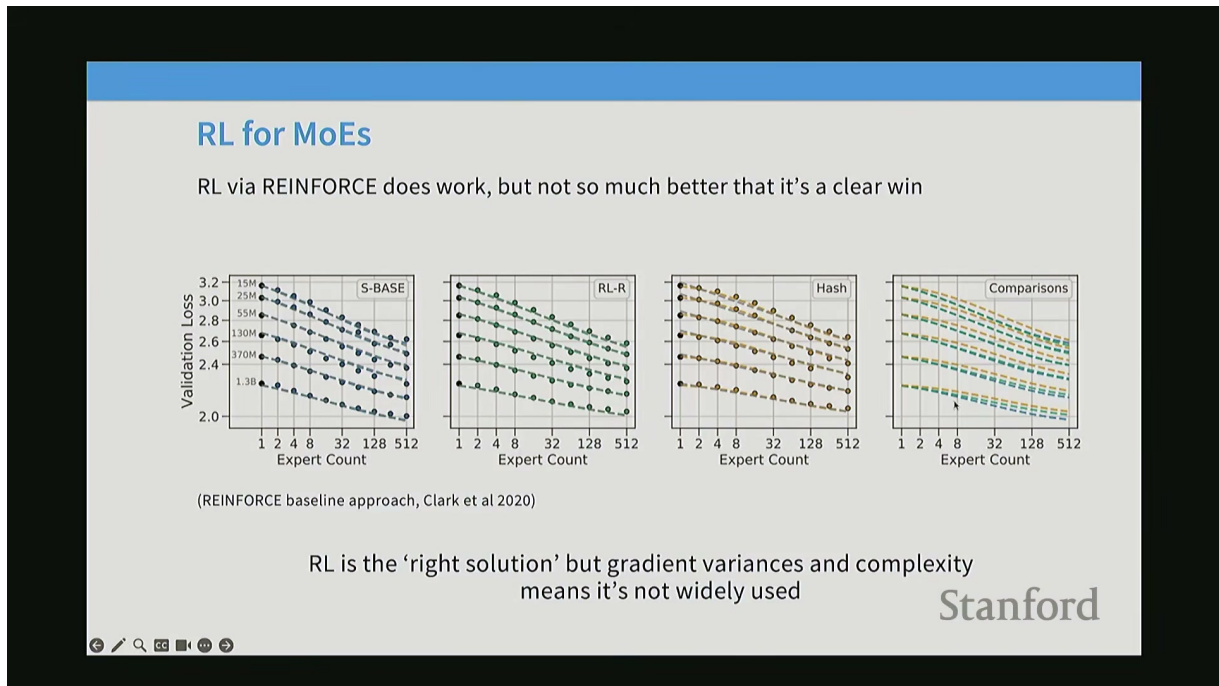


图 16: REINFORCE 路由可工作，但课件中的 baseline 并未明显优于简单哈希。

视频画面时间区间: 00:43:10–00:43:50。

“可优化”不等于“值得部署”

RL 路由只解决离散梯度估计，并不会自动满足设备负载、通信上限或推理确定性。若训练收益不足以覆盖方差、调参与系统复杂度，简单 top- k 加辅助约束往往更可靠。

5.2 随机扰动与 router jitter

一种早期方案给线性 affinity 加入输入相关的高斯噪声：

$$s_{t,i} = w_i^\top x_t + \epsilon_{t,i} \text{softplus}(v_i^\top x_t), \quad \epsilon_{t,i} \sim \mathcal{N}(0, 1).$$

- v_i : 控制专家 i 噪声尺度的可学习向量；
- $\epsilon_{t,i}$: 标准高斯随机变量；
- softplus: 保证噪声尺度为正。

更简单的 jitter 直接对输入或 logits 施加小幅乘性噪声。它让靠近选择边界的 token 在训练时探索不同专家，降低路由器过早锁死的风险。

Stochastic approximations

$$G(x) = \text{Softmax}(\text{KeepTopK}(H(x), k))$$

$$H(x)_i = (x \cdot W_g)_i + \text{StandardNormal}() \cdot \text{Softplus}((x \cdot W_{\text{noise}})_i)$$

$$\text{KeepTopK}(v, k)_i = \begin{cases} v_i & \text{if } v_i \text{ is in the top } k \text{ elements of } v. \\ -\infty & \text{otherwise.} \end{cases}$$

From Shazeer et al 2017 – routing decisions are *stochastic* with gaussian perturbations.

1. This naturally leads to experts that are a bit more robust.
2. The softmax means that the model learns how to rank K experts

Stanford

图 17: Shazeer 等工作的可学习高斯扰动路由。

视频画面时间区间: 00:44:25–00:45:05。

```

1 def router_jitter(x, eps=0.01, training=True):
2     if training:
3         noise = uniform(1.0 - eps, 1.0 + eps, shape=x.shape)
4         x = x * noise
5     logits = x @ router_weight
6     expert_ids = topk(logits, k)
7     return expert_ids, logits

```

Listing 1: 训练期 router jitter 的最小化伪代码

Stochastic approximations

```

if is_training:
    # Add noise for exploration across experts.
    router_logits += mtf.random_uniform(shape=router_logits.shape, minval=1-eps, maxval=1+eps)

# Convert input to softmax operation from bfloat16 to float32 for stability.
router_logits = mtf.to_float32(router_logits)

# Probabilities for each token of what expert it should be sent to.
router_probs = mtf.softmax(router_logits, axis=-1)

```

Stochastic jitter in Fedus et al 2022. This does a uniform multiplicative perturbation for the same goal of getting less brittle experts. This was later removed in Zoph et al 2022

Method	Fraction Stable	Quality (↑)
Baseline	4/6	-1.755 ±0.02
Input jitter (10 ⁻²)	3/3	-1.777 ±0.03
Dropout (0.1)	3/3	-1.822 ±0.11

Stanford

图 18: 乘性 jitter、dropout 实现与课件消融结果。

视频画面时间区间: 00:45:40–00:46:20。

随机性要有明确退出路径

训练期噪声应在评估和推理时关闭；否则同一请求可能产生不同路由，破坏缓存、复现和容量规划。验证时还应单独检查“关闭噪声后的负载”，因为训练期平均平衡不保证确定性推理仍平衡。

5.3 本章小结

离散路由可以用 RL 直接估计梯度，也可以用随机扰动探索边界；但大规模 MoE 最常见的仍是普通 top- k 加启发式负载控制。选择训练方法时，除了最终 loss，还要比较梯度方差、路由稳定性、负载尾部与推理确定性。

6 负载平衡：从辅助损失到在线偏置

6.1 Switch 风格的专家平衡

对于一个含 T 个 token 的批次，专家 i 的实际分配频率定义为

$$f_i = \frac{1}{T} \sum_{t=1}^T \mathbf{1}[i \in S_t].$$

- f_i : 专家 i 实际处理 token 的频率；
- T : 当前统计窗口的 token 数；
- 若使用 top- k ，可按实现约定再除以 k 以归一化。

路由器对专家 i 的平均软概率为

$$P_i = \frac{1}{T} \sum_{t=1}^T p_{t,i}.$$

- P_i : 路由器“想要”选择专家 i 的平均程度;
- 与离散的 f_i 不同, P_i 对 logits 可微;
- P_i 提供辅助目标能传回路由器的梯度通道。

Switch Transformer 风格的平衡损失常写成

$$\mathcal{L}_{\text{bal}} = \alpha N \sum_{i=1}^N f_i P_i.$$

- α : 辅助损失权重;
- N : 专家数, 用于使损失尺度对专家数量更稳定;
- 均匀分配时 f_i 与 P_i 都接近 $1/N$ 。

Heuristic balancing losses

Another key issue – systems efficiency requires that we use experts evenly..

For each Switch layer, this auxiliary loss is added to the total model loss during training. Given N experts indexed by $i = 1$ to N and a batch B with T tokens, the auxiliary loss is computed as the scaled dot-product between vectors f and P ,

$$\text{loss} = \alpha \cdot N \cdot \sum_{i=1}^N f_i \cdot P_i \quad (4)$$

where f_i is the fraction of tokens dispatched to expert i ,

$$f_i = \frac{1}{T} \sum_{x \in B} \mathbb{1}\{\text{argmax } p(x) = i\} \quad (5)$$

and P_i is the fraction of the router probability allocated for expert i ,²

$$P_i = \frac{1}{T} \sum_{x \in B} p_i(x). \quad (6)$$

From the Switch Transformer [Fedus et al 2022]

The derivative with respect to $p_i(x)$ is $\frac{\alpha N}{T^2} \sum \mathbb{1}_{\text{argmax } p(x)=i}$,
so more frequent use = stronger downweighting

Stanford

图 19: Switch 辅助损失同时使用离散分配频率 f_i 和可微概率 P_i 。

视频画面时间区间: 00:49:25–00:50:05。

辅助损失的冲突

平衡损失优化的是资源使用, 不是直接优化 next-token likelihood。权重太小会造成专家塌缩和 token 丢弃, 太大则可能强迫语义不合适的 token 均匀分流, 损害主任务质量。它必须与容量、批次规模和并行拓扑一起调。

6.2 从专家平衡扩展到设备平衡

若每个设备包含多个专家，仅让专家频率平均仍不一定让设备通信平均。可把设备 q 的平均路由量聚合为

$$F_q = \sum_{i \in \mathcal{E}_q} f_i, \quad Q_q = \sum_{i \in \mathcal{E}_q} P_i, \quad \mathcal{L}_{\text{device}} = \beta D \sum_{q=1}^D F_q Q_q.$$

- F_q, Q_q : 设备 q 的离散负载和软路由质量;
- D : 专家并行设备数;
- β : 设备级平衡损失权重。

Example from deepseek (v1-2)

Per-expert balancing – same as the switch transformer

$$\mathcal{L}_{\text{ExpBal}} = \alpha_1 \sum_{i=1}^{N'} f_i P_i, \quad (12)$$

$$f_i = \frac{N'}{K'T} \sum_{t=1}^T \mathbb{I}(\text{Token } t \text{ selects Expert } i), \quad (13)$$

$$P_i = \frac{1}{T} \sum_{t=1}^T s_{i,t}, \quad (14)$$

Per-device balancing – the objective above, but aggregated by device.

$$\mathcal{L}_{\text{DevBal}} = \alpha_2 \sum_{i=1}^D f'_i P'_i, \quad (15)$$

$$f'_i = \frac{1}{|\mathcal{E}_i|} \sum_{j \in \mathcal{E}_i} f_j, \quad (16)$$

$$P'_i = \sum_{j \in \mathcal{E}_i} P_j, \quad (17)$$

Stanford

图 20: DeepSeek V1/V2 同时约束专家级与设备级负载。

视频画面时间区间: 00:50:40–00:51:20。

6.3 DeepSeek-V3 的辅助损失自由偏置

另一条路线不把平衡项直接加进主训练目标，而为每个专家维护路由偏置 b_i 。选择时使用修正分数 $s_{t,i} + b_i$ ，混合权重仍可使用原始 **affinity**。统计到专家过载后降低其偏置，欠载则提高：

$$b_i^{(r+1)} = b_i^{(r)} + \gamma \operatorname{sign}(\bar{f} - f_i^{(r)}).$$

- r : 偏置控制器的更新步;
- $\bar{f}$: 目标平均负载;
- γ : 每次调整的步长;
- 过载时 $\bar{f} - f_i < 0$ ，偏置下降，专家变得不易被选。

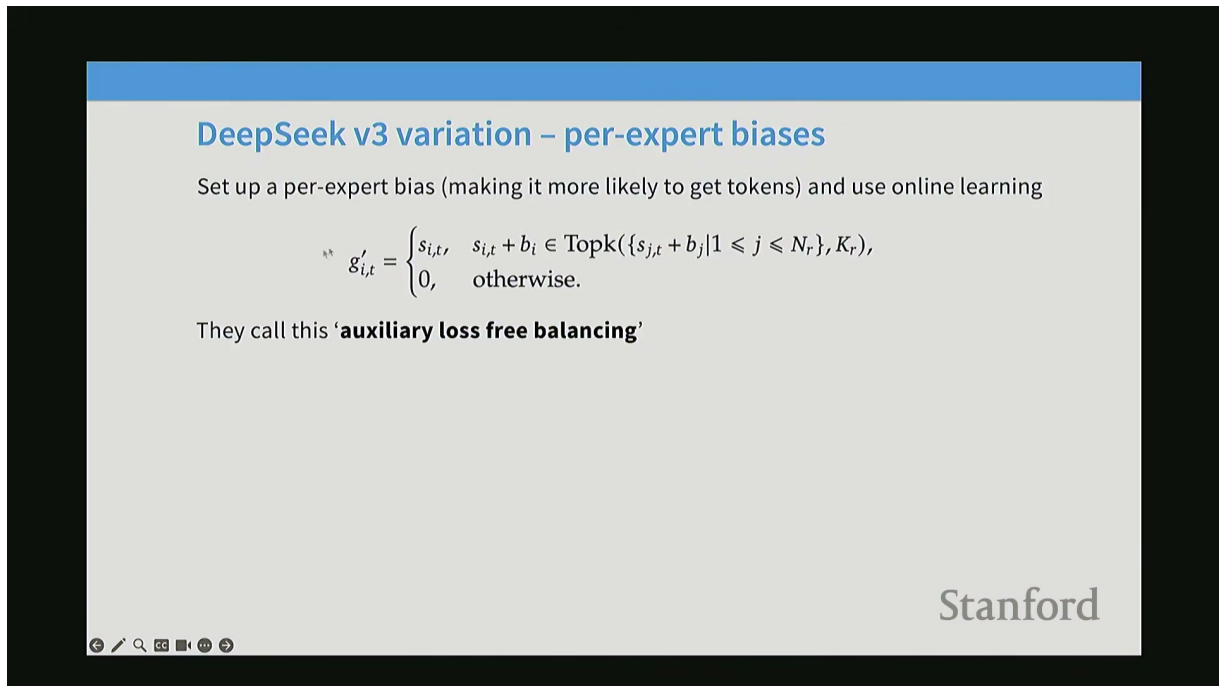


图 21: 在线更新专家偏置, 实现不直接污染主损失的负载控制。

视频画面时间区间: 00:53:10–00:53:50。

把它看成控制器, 而非普通梯度参数

b_i 的任务是校正资源使用, 不是表达 token 与专家的语义匹配。更新太快会产生负载振荡, 太慢则追不上数据分布漂移。实践中应监控专家利用率直方图、最大/平均负载比、跨设备字节数和偏置轨迹。

6.4 本章小结

经典辅助损失通过 $f_i P_i$ 给路由器可微的平衡信号, 并可从专家级扩展到设备级; 辅助损失自由方法则把负载平衡作为在线偏置控制问题。两者都不能只凭平均值验收, 还需检查最热专家、最慢设备和推理分布变化。

7 系统实现、容量与数值稳定性

7.1 Expert parallelism 的数据流

一次 MoE 前向通常包含: 计算 router logits、TopK 选专家、按目标专家排列 token、all-to-all 发送、执行专家 GEMM、all-to-all 返回、按原顺序合并。若每个 token 表示占 b_h 字节, 每个 token 被复制到 k 个专家, 粗略单向通信量为

$$V_{\text{comm}} \approx T k b_h.$$

- V_{comm} : 一次派发或返回阶段的有效载荷量;
- T : 参与本次 MoE 层计算的 token 数;

- b_h : 一个隐藏向量的字节数;
- 实际通信还包含 padding、索引和协议开销。

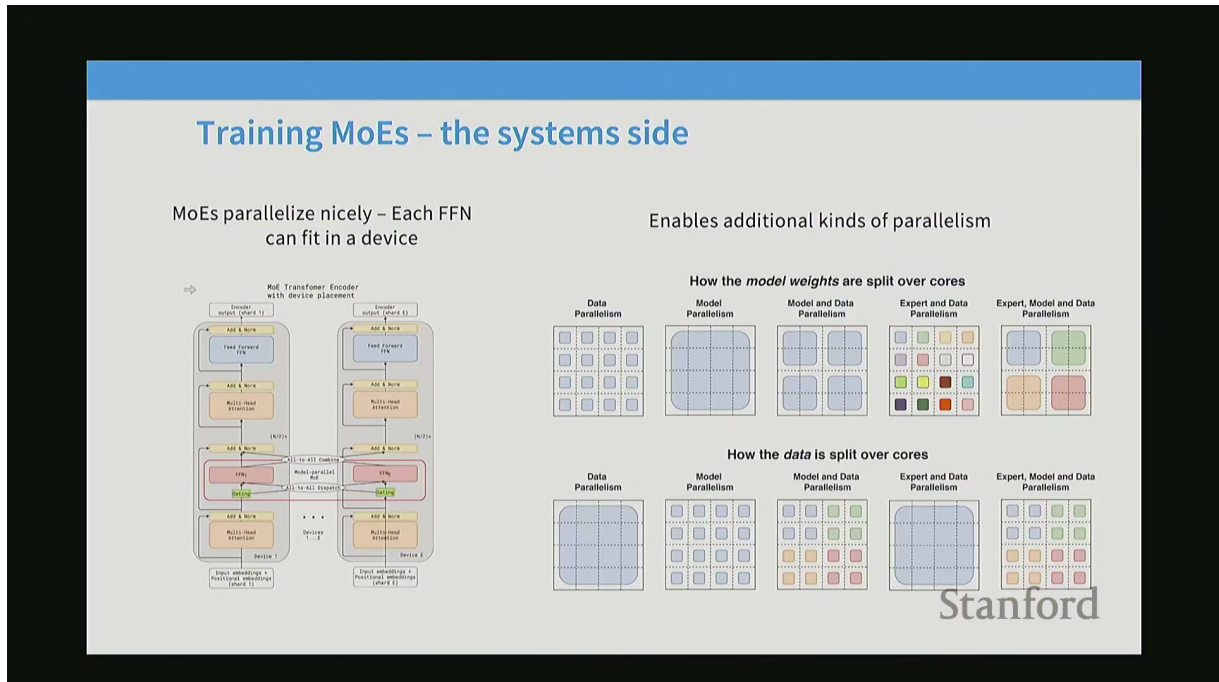


图 22: 专家并行可与数据并行、张量并行等组合。

视频画面时间区间: 01:00:40–01:01:20。

系统优化目标是“大而规则的专家批次”

GPU 擅长大矩阵乘法，不擅长成百上千个大小不一的小调用。实现需要将发往同一专家的 token 聚合成连续块，使用 grouped GEMM 或 MegaBlocks 一类稀疏 kernel，并尽量让通信与计算重叠。

7.2 容量因子与 token dropping

若每个 token 激活 k 个专家，理想均匀负载约为 Tk/N 。给定容量因子 $c > 1$ ，单专家容量可设为

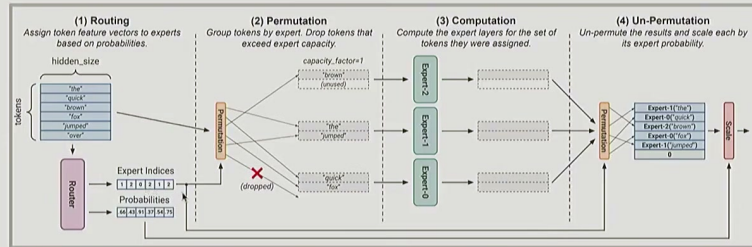
$$C_{\text{expert}} = \left\lceil c \frac{Tk}{N} \right\rceil.$$

- C_{expert} : 本批次一个专家最多接收的 token 数;
- c : 为路由不均衡预留的冗余比例;
- 超过容量的 token 可能被丢弃、改送下一专家或走残差旁路。

Fun side issue – stochasticity of MoE models

There was speculation that GPT-4's stochasticity was due to MoE..

Why would a MoE have additional randomness?



Token dropping from routing happens at a *batch* level – this means that other people's queries can drop your token!

Stanford

图 23: 批量路由、重排、专家计算与逆重排；过载会引入 token dropping。

视频画面时间区间：01:03:10–01:03:50。

单批次负载有统计波动。若把 token 到某专家的选择粗略看成概率为 p_i 的伯努利事件，则

$$\text{Var}[n_i] \approx Tk p_i(1 - p_i).$$

- n_i : 专家 i 在批次中的 token 数；
- p_i : 该专家的平均被选概率；
- 批次越小，相对波动越大，越容易触发容量上限。

训练期“偶尔丢 token”会变成数据相关噪声

丢弃不是均匀随机的：最热门专家对应的 token 更容易被丢。它会改变模型看到的有效计算图，也会使批次大小、样本混合和设备拓扑影响最终质量。应记录 dropped-token rate，并对领域、长度与语言切片检查偏差。

7.3 路由器为什么常用 float32

TopK 对排序边界极敏感。bfloat16 的尾数较短，两个接近的 logits 可能被舍入到相同或相反顺序；softmax 又会放大相对差异。因此常见做法是：主体专家计算用 bfloat16，router logits、softmax 和部分归一化使用 float32。

稳定性辅助项 z-loss 常写为

$$\mathcal{L}_z = \lambda_z \frac{1}{T} \sum_{t=1}^T \left(\log \sum_{i=1}^N e^{s_{t,i}} \right)^2.$$

- λ_z : z-loss 权重；

- $s_{t,i}$: 路由 logits;
- 该项抑制 logits 整体尺度漂移, 降低 softmax 饱和与数值尖峰。

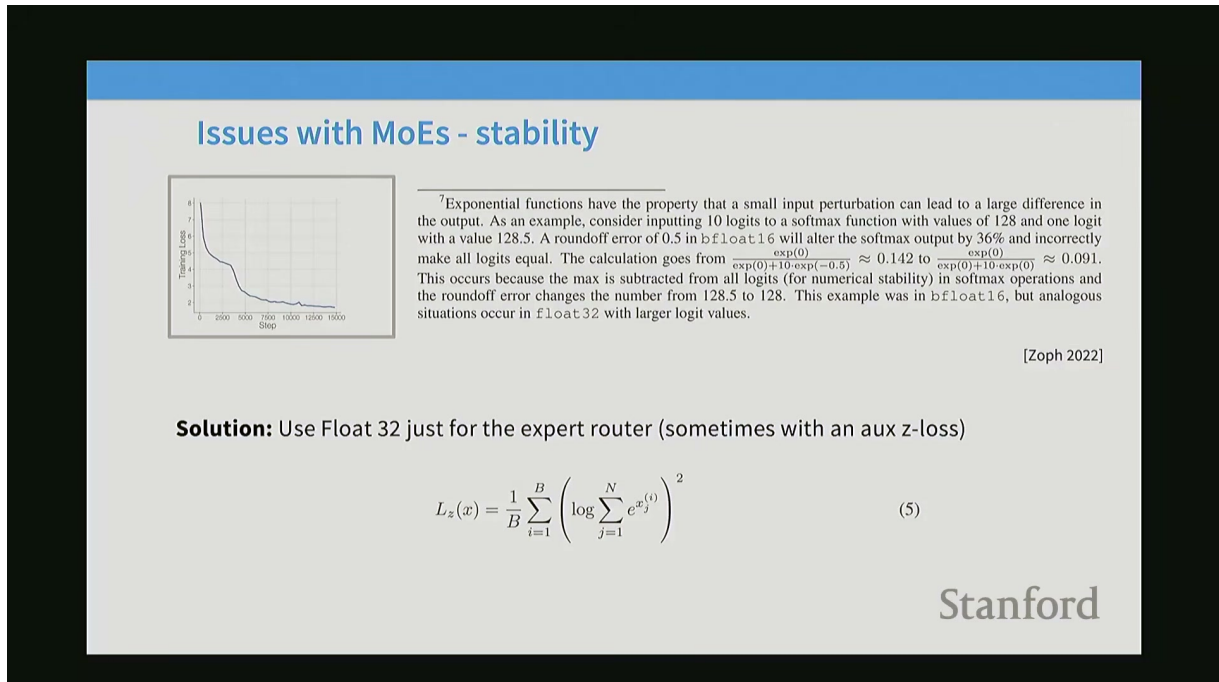


图 24: bfloat16 舍入可显著改变 softmax; router 常用 float32 并加入 z-loss。

视频画面时间区间: 01:05:40–01:06:20。

softmax 对 logit 的局部敏感性由雅可比给出:

$$\frac{\partial p_i}{\partial s_j} = p_i(\mathbf{1}[i = j] - p_j).$$

- p_i : softmax 后第 i 个专家的概率;
- s_j : 第 j 个专家的 logit;
- 当两个专家得分接近时, 小扰动可能改变 TopK 的离散成员。

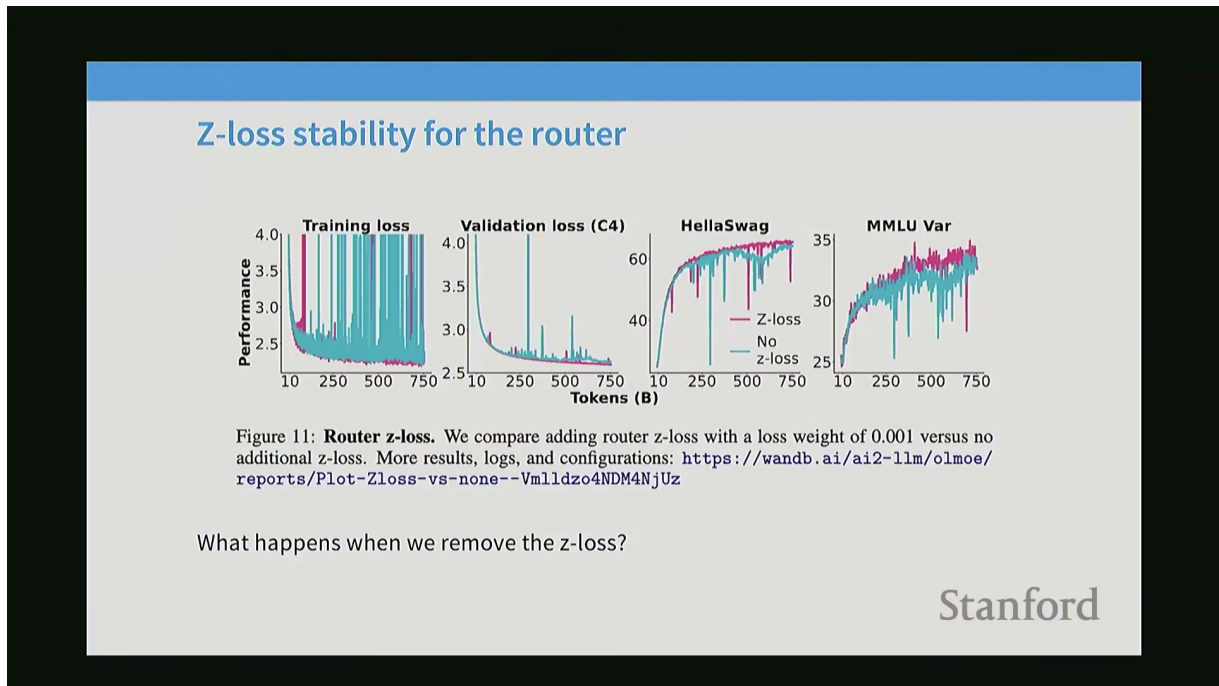


图 25: 移除 z-loss 后出现显著验证损失尖峰，说明路由数值稳定性会影响整体训练。

视频画面时间区间: 01:06:10–01:06:50。

7.4 本章小结

高性能 MoE 是路由、通信与 grouped GEMM 的协同系统。容量因子缓冲批次波动，却可能引入数据相关的 token dropping；float32 router 和 z-loss 则保护离散选择免受数值误差放大。系统验收必须同时报告质量、吞吐、通信、丢弃率和负载尾部。

8 训练策略与 DeepSeek 系列演进

8.1 从稠密模型 upcycle 成 MoE

从零训练不是唯一入口。Upcycling 复制一个已训练稠密 FFN 的权重到多个专家，再用小噪声打破对称性：

$$W_i^{(0)} = W_{\text{dense}} + \varepsilon_i.$$

- $W_i^{(0)}$: 专家 i 的初始化权重；
- W_{dense} : 原稠密 FFN 权重；
- ε_i : 小幅随机扰动，防止所有专家永远保持完全相同。

初始化时各专家能力相近，可直接继承稠密模型的知识；继续预训练后，路由和专家逐步分化。它能减少从头训练成本，但如果后续 token 预算太少，专家可能来不及形成有效分工；复制全部专家也会立刻扩大 checkpoint 和服务存储。

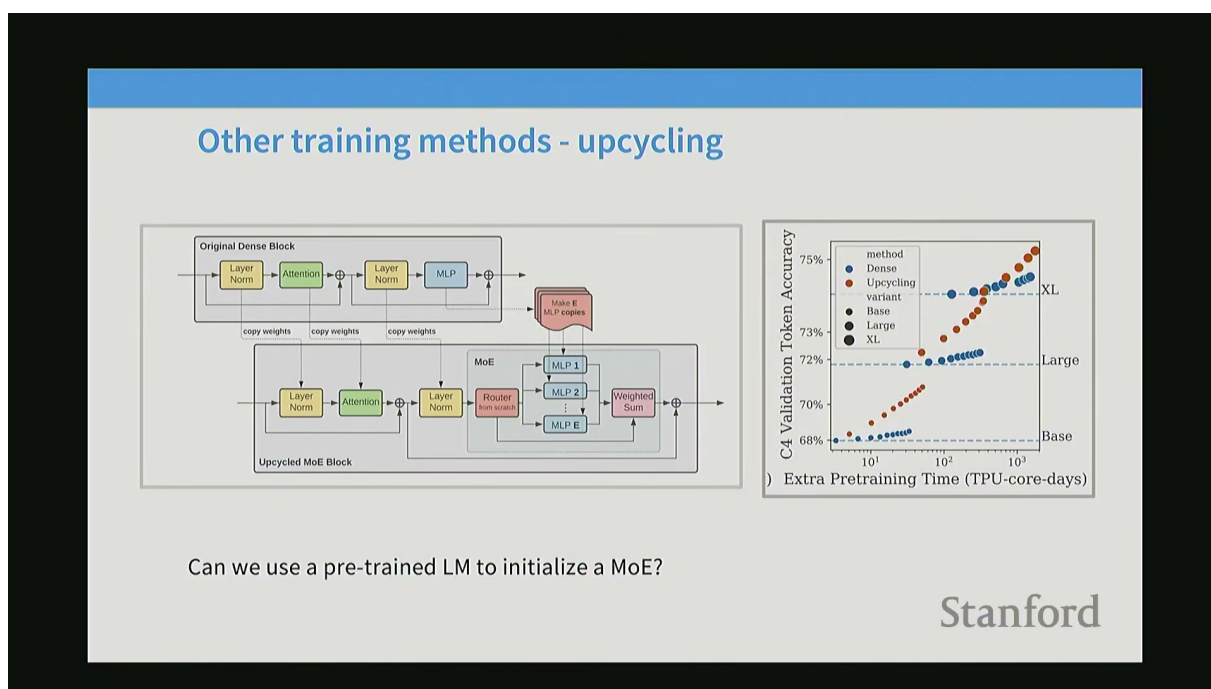


图 26: Upcycling: 复制稠密 FFN 构造专家, 再通过继续预训练促使分化。
视频画面时间区间: 01:08:40-01:09:20。

微调阶段要防专家过拟合

MoE 总参数巨大, 而指令微调数据往往远小于预训练数据。只用小规模 SFT 可能让少数专家快速记忆样本。可采用更小学习率、冻结部分专家、混入预训练数据, 或先在大规模多样化 SFT 数据上训练, 并监控路由熵与专家利用率。

8.2 DeepSeekMoE V1: 细粒度与共享专家

V1 把结构创新集中在专家层: 约 16B 总参数、2.8B 激活参数; 使用 2 个共享专家以及 64 个细粒度路由专家, 每个 token 选择约 6 个路由专家。训练仍采用标准 top- k , 并结合专家级与设备级辅助平衡。

DeepSeek MoE v1-v2-v3

To wrap up, we'll walk through the DeepSeek MoE architecture.

V1 (16B – 2.8 active):

Shared (2) + Fine-grained (64/4) experts

Transformer Block $\times L$

Feed-Forward Network

RMS Norm

DeepSeekMoE

Standard, top-k routing

$$h'_i = u_i + \sum_{j=1}^{N_i} \text{FFN}_i^{(j)}(u_i) + \sum_{j=1}^{N_i} g_{i,j} \text{FFN}_i^{(j)}(u_i),$$

$$g_{i,j} = \begin{cases} s_{i,j}, & s_{i,j} \in \text{Topk}(\{s_{i,j} | 1 \leq j \leq N_i\}, K), \\ 0, & \text{otherwise}, \end{cases}$$

$$s_{i,j} = \text{Softmax}_j(u_i^T e_j),$$

Standard Aux-loss balancing (Expert + Device)

$$\mathcal{L}_{\text{Expert}} = a_i \sum_{j=1}^{N_i} f_j P_{i,j}$$

$$f_i = \frac{N_i}{K \cdot T} \sum_{t=1}^T \mathbb{1}(\text{Token } t \text{ selects Expert } i),$$

$$P_i = \frac{1}{T} \sum_{t=1}^T s_{i,t},$$

Stanford

图 27: DeepSeekMoE V1: 细粒度专家、共享专家与双层负载平衡。

视频画面时间区间: 01:10:55–01:11:35。

8.3 DeepSeek-V2: Top-M 设备约束通信

V2 扩展到约 236B 总参数、21B 激活参数，使用 2 个共享专家和 160 个细粒度专家，并激活其中 6 个。大规模下，单纯选全局 top-k 可能把一个 token 发往许多设备。Top-M device routing 先把候选限制到少数设备，再在其中选专家，直接控制通信扇出。

若 token t 访问的设备集合为 $\mathcal{D}_t$ ，Top-M 约束为

$$|\mathcal{D}_t| \leq M.$$

- $\mathcal{D}_t$: 承载该 token 所选专家的设备集合;
- M : 允许访问的最大设备数;
- 该约束减少网络目的地数，但可能牺牲全局最优专家组合。

DeepSeek MoE v2

V2 (236B – 21 active):

Shared (2) + Fine-grained (160/10) experts, 6 active

New things:

Top-M device routing

For DeepSeek-V2, beyond the naive top-K selection of routed experts, we additionally ensure that the target experts of each token will be distributed on at most M devices. To be specific, for each token, we first select M devices that have experts with the highest affinity scores in them. Then, we perform top-K selection among experts on these M devices. In practice, we find that when $M \geq 3$, the device-limited routing can achieve a good performance roughly aligned with the unrestricted top-K routing.

Communication balancing loss – balancing both communication in and out

$$\mathcal{L}_{\text{CommBal}} = \alpha_2 \sum_{i=1}^M p_i^{\text{in}} p_i^{\text{out}}, \quad (29)$$

$$p_i^{\text{in}} = \frac{1}{M} \sum_{t=1}^T \mathbb{1}(\text{Token } t \text{ is sent to Device } i), \quad (30)$$

$$p_i^{\text{out}} = \sum_{t=1}^T p_{i,t} \quad (31)$$

where α_2 is a hyper-parameter called communication balance factor. The device-limited routing mechanism operates on the principle of ensuring that each device transmits at most M hidden states to other devices. Simultaneously, the communication balance loss is employed to encourage each device to receive around M hidden states from other devices. The communication balance loss guarantees a balanced exchange of information among devices, promoting efficient communications.

图 28: DeepSeek-V2: 模型扩大后用 Top-M device routing 控制通信扇出。

视频画面时间区间: 01:13:25–01:14:05。

通信约束应进入路由设计，而不是事后补救

当模型扩展到数百亿激活参数时，网络拓扑已经是模型的一部分。把所有专家视为等距离、先路由再观察通信，通常会得到不可部署的方案。Top-M 体现了“质量目标与硬件约束共同定义路由空间”。

8.4 DeepSeek-V3: 平衡控制与推理鲁棒性

V3 约 671B 总参数、37B 激活参数，采用 1 个共享专家、256 个路由专家并激活 8 个。其路由组合包括 sigmoid affinity、选中权重归一化、Top-M 设备约束和辅助损失自由偏置；同时保留较小的序列级平衡约束，以防单条推理序列把少数专家压垮。

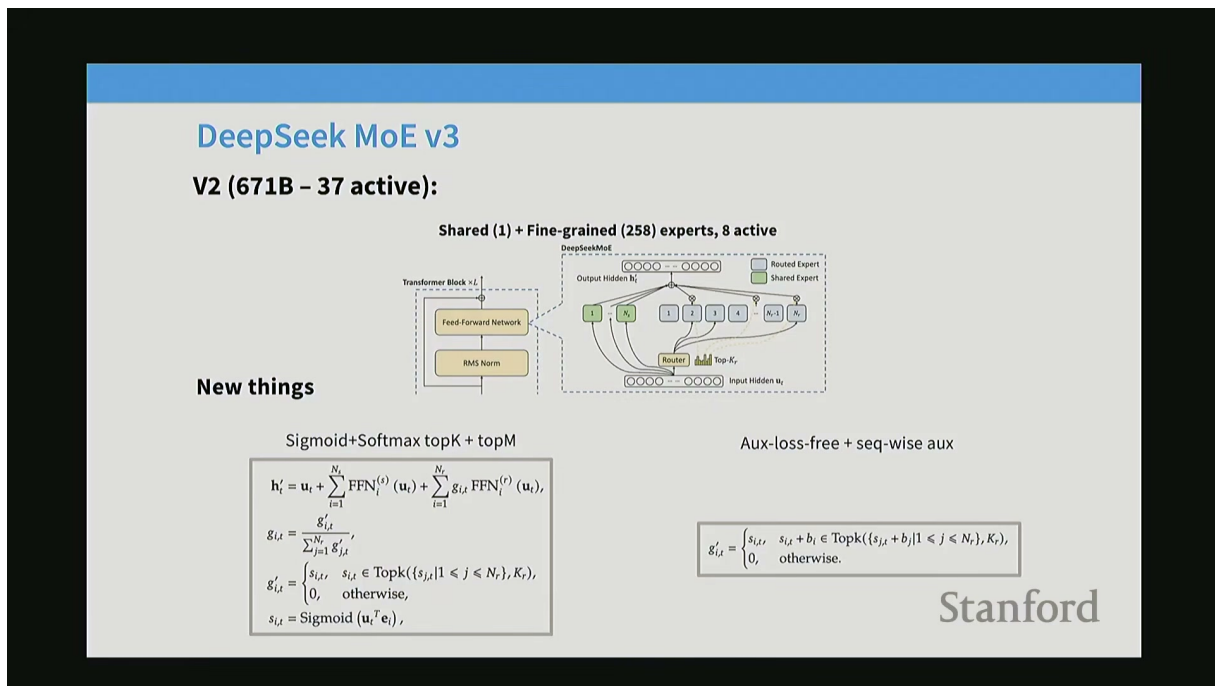


图 29: DeepSeek-V3: 更大专家系统、辅助损失自由平衡与序列级防护。

视频画面时间区间: 01:15:40–01:16:20。

课件标题笔误与口头内容要分开核对

该页架构内容对应 V3 的 671B/37B-active 配置，但画面标题可能残留“V2”字样。讲义按上下文和数值归入 V3，并在数值主张清单中保留核验记录，避免把版面笔误传播成模型事实。

8.5 本章小结

Upcycling 让稠密 checkpoint 成为 MoE 的低成本起点。DeepSeek 系列的演进不是单纯堆参数：V1 引入细粒度和共享专家，V2 让通信拓扑进入路由约束，V3 将负载控制从主损失中分离，并增加面向单序列推理的防护。这条路线展示了算法、系统和部署约束的共同演化。

9 MLA、MTP 与全课总结

9.1 MLA: 把 KV cache 压缩到低维潜变量

Multi-head Latent Attention (MLA) 先把隐藏状态压缩为低维 KV 潜变量：

$$c_t^{KV} = W^{DKV} h_t.$$

- h_t : 位置 t 的隐藏状态;
- W^{DKV} : KV 下投影矩阵;
- c_t^{KV} : 缓存的低维潜变量, 维度远小于完整多头 K/V。

需要注意力键和值时, 再通过上投影恢复:

$$k_t^C = W^{UK} c_t^{KV}, \quad v_t^C = W^{UV} c_t^{KV}.$$

- W^{UK}, W^{UV} : 键和值的上投影;
- k_t^C, v_t^C : 由压缩潜变量恢复的内容部分;
- 推理时只缓存 c_t^{KV} , 显著降低长上下文 KV cache。

为了避免显式恢复键带来的额外 FLOPs, 可把 W^{UK} 吸收到 query 投影或后续点积中。但 RoPE 是位置相关变换, 不能简单与固定矩阵交换, 因此 DeepSeek 保留一小部分不压缩、专门承载 RoPE 的 key/query 维度。

What else do you need to make DeepSeek MoE v3?

Basic idea: express the Q, K, V as functions of a lower-dim, 'latent' activation

$$c_t^{KV} = W^{DKV} h_t,$$

$$k_t^C = W^{UK} c_t^{KV},$$

$$v_t^C = W^{UV} c_t^{KV},$$

Benefits: when KV-caching, we only need to store c_t^{KV} , which can be much smaller.
 W^{UK} can be merged into the Q projection
(they also compress queries, for memory savings during training)

$c_t^Q = W^{DQ} h_t,$
 $q_t^C = W^{UQ} c_t^Q,$

Complexity: rope conflicts with MLA-style caching.

Without RoPE - $\langle Q, K \rangle = \langle h W^Q, W^{UK} c_t^{KV} \rangle = \langle h W^Q W^{UK}, c_t^{KV} \rangle$

With RoPE - $\langle Q R_q, R_k K \rangle = \langle h W^Q R_q, R_k W^{UK} c_t^{KV} \rangle = \langle h W^Q R_q R_k W^{UK}, c_t^{KV} \rangle$

The solution - Have a few non-latent key dimensions that can be rotated

图 30: MLA: 低维 KV cache、矩阵吸收以及 RoPE 阻止完全吸收的问题。

视频画面时间区间: 01:18:10-01:18:50。

为什么在 MoE 课末讨论 MLA

MoE 主要降低每个 token 的参数计算, MLA 主要降低自回归推理的 KV cache 与内存带宽。二者解决不同瓶颈, 却共同服务于“总模型很大、单 token 成本可控”的目标, 也是 DeepSeek-V2/V3 系统效率的两根支柱。

9.2 MTP: 同时预测多个未来 token

标准语言模型只预测下一 token。Multi-Token Prediction (MTP) 增加轻量模块, 同时预测更远位置。若预测深度为 D , 可写成

$$\mathcal{L}_{\text{MTP}} = \frac{1}{D} \sum_{d=1}^D \text{CE}(p_{\theta}(x_{t+d} \mid x_{\leq t}, d), x_{t+d}).$$

- D : 额外预测的未来深度;
- CE: 交叉熵;
- $p_{\theta}(\cdot, d)$: 第 d 个预测头或轻量预测模块;

- 本讲介绍的实际配方只向前多预测一个 token。

总训练目标可写为

$$\mathcal{L} = \mathcal{L}_{\text{NTP}} + \lambda_{\text{MTP}} \mathcal{L}_{\text{MTP}} + \mathcal{L}_{\text{router}}.$$

- $\mathcal{L}_{\text{NTP}}$: 标准 next-token prediction 损失;
- λ_{MTP} : MTP 辅助目标权重;
- $\mathcal{L}_{\text{router}}$: 可能包含平衡或 z-loss; 采用辅助损失自由平衡时其中不含专家平衡项。

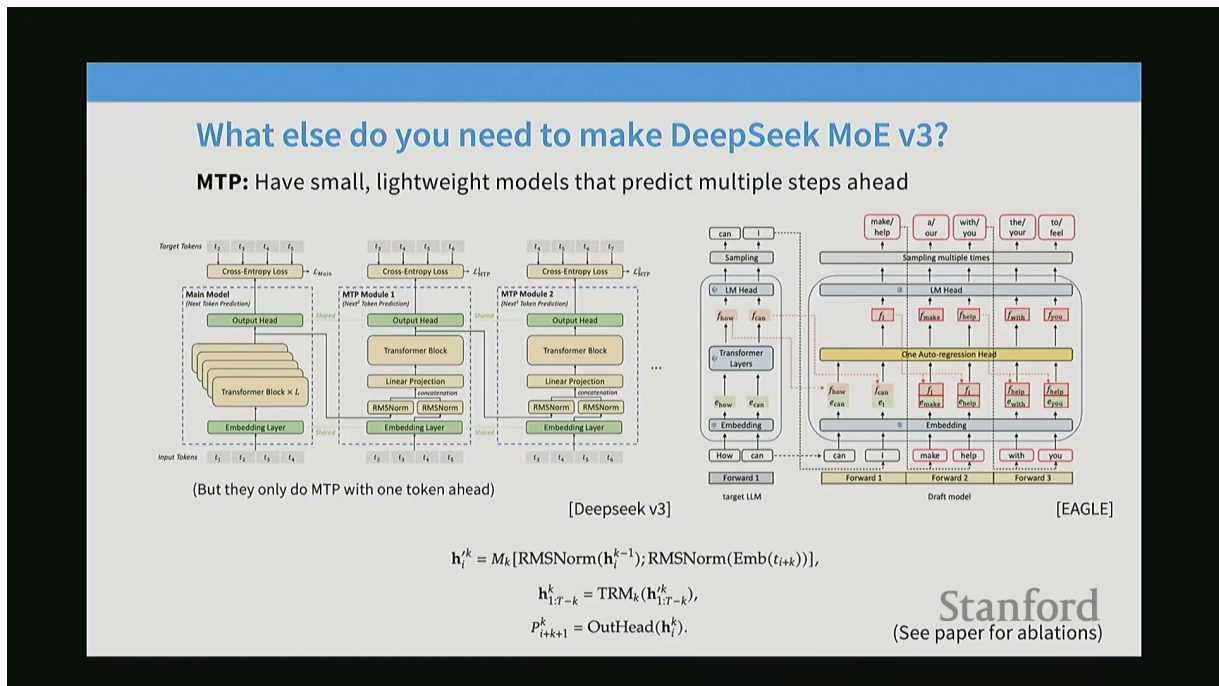


图 31: MTP: 用轻量一层模块增加第二个未来 token 的预测信号。

视频画面时间区间: 01:20:25-01:21:05。

MTP 的价值不只在“多一个标签”

更远 token 的监督迫使表示包含更具前瞻性的结构信息，也可为 speculative decoding 提供候选。但辅助头过重会抵消收益，训练权重过大则可能干扰标准下一 token 目标，因此通常采用轻量模块和有限预测深度。

9.3 把全讲串成一条工程链

一个可工作的 MoE 系统可以沿以下顺序设计:

1. 先确定目标激活 FLOPs、总参数预算和部署节点;
2. 选择 FFN 专家粒度、共享专家数与 top-k;
3. 将设备拓扑写进 expert placement 与路由约束;
4. 选择辅助损失或在线偏置, 监控专家与设备负载尾部;

5. 使用 grouped GEMM、通信重叠、容量控制和 dropped-token 观测；
6. 用 float32 router、z-loss 与推理序列压力测试保护稳定性；
7. 最后联合评估 loss、下游质量、吞吐、显存、通信和尾延迟。

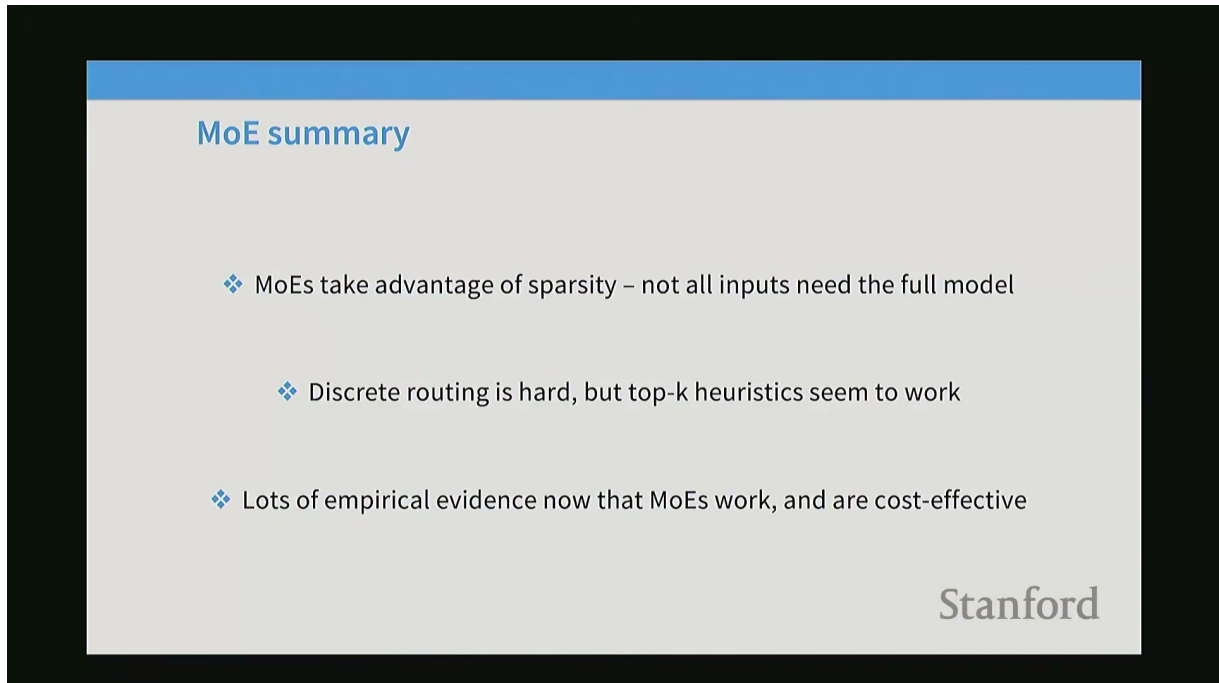


图 32: 本讲结论: 稀疏性带来强实证收益, top- k 和启发式训练构成当前主流。
视频画面时间区间: 01:21:25–01:22:04。

最终判断

在 FLOP 受限的语言模型训练中, MoE 已有充分实证优势; 困难不在“稀疏是否有用”, 而在如何稳定地选择专家、均衡硬件负载并让理论计算节省兑现为端到端速度。高质量 MoE 不是一个孤立层, 而是一套算法与系统共同设计。

9.4 复习检查表

- 能否区分总参数、激活参数、理论 FLOPs 与实际吞吐？
- 能否从 router logits 推导 softmax、TopK、混合输出与梯度断点？
- 能否解释细粒度专家和共享专家分别解决什么问题？
- 能否写出 $f_i P_i$ 平衡损失, 并说明它为何可能损伤主任务？
- 能否说明辅助损失自由偏置为何更像在线控制器？
- 能否画出 route-permute-all-to-all-GEMM-unpermute 数据流？
- 能否解释 token dropping、float32 router 和 z-loss 的必要性？
- 能否概括 DeepSeek V1、V2、V3 在结构、通信和平衡上的演进？
- 能否说明 MLA 与 MoE 分别优化推理中的哪一种瓶颈？

9.5 本章小结

MLA 压缩 KV cache, MTP 增加前瞻监督, MoE 稀疏化参数计算; 三者共同构成 DeepSeek 系列效率设计的一部分。本讲最重要的方法论, 是不要把架构创新与系统现实拆开: 路由公式、专家形状、网络拓扑、数值精度和服务分布必须作为一个整体验收。